

A Field Reference Guide to Evaluating Machine Learning Systems

A model's own accuracy number is rarely the whole story. This guide collects the metrics, tests, and failure modes that separate a genuinely measured system from one that only looks measured, organized by the kind of task being evaluated.

It is organized to be read start to end once and then used as a reference afterward: each section stands on its own, the metric reference table and the significance test table are both meant to be looked up directly without reading the surrounding prose again, and the glossary at the end collects every term used loosely elsewhere in one place with a precise definition.

Classification Metrics

Accuracy is the fraction of predictions that match the true label. It is intuitive and easy to explain, and it is also the metric most likely to mislead on an imbalanced dataset: a model that always predicts the majority class can post a high accuracy while being useless for the minority class that usually matters most.

Precision is the fraction of positive predictions that were actually correct; recall is the fraction of actual positives the model found. The two trade off against each other as a decision threshold moves, and which one matters more depends entirely on whether a false positive or a false negative is more costly in the system being built.

The F1 score is the harmonic mean of precision and recall, useful as a single number when both errors matter roughly equally. The harmonic mean, rather than a simple average, punishes a system that is excellent on one of the two and poor on the other.

ROC-AUC measures ranking quality across every possible decision threshold at once, which makes it threshold independent but also makes it insensitive to class imbalance in a way that can overstate performance; PR-AUC is generally the more honest choice when the positive class is rare.

Choosing a Decision Threshold

Most classifiers output a probability, not a label, and the label only appears once that probability is compared against a threshold, conventionally one half by default and almost never the right choice for a specific application. Moving the threshold trades precision against recall directly, and the right point on that curve depends on the relative cost of a false positive against a false negative in the system the classifier actually feeds into.

A threshold picked once during development and never revisited quietly assumes the cost tradeoff that justified it still holds, which is worth checking explicitly rather than assuming, especially after any change to the population the classifier is being applied to.

Regression Metrics

Mean absolute error reports the average size of a prediction's error in the target's own units, and treats every unit of error the same regardless of size. Root mean squared error squares each error before averaging, which means a few large misses dominate the number even if most predictions were close.

R-squared reports the fraction of variance in the target explained by the model, relative to a baseline that always predicts the mean. It is easy to misread: a low R-squared does not always mean a bad model, since some targets are simply noisy and hard to predict from any set of features.

Ranking and Retrieval Metrics

Precision at k and recall at k score a ranked list against a fixed cutoff: precision at k asks what fraction of the top k results are relevant, and recall at k asks what fraction of all relevant items were found within the top k. Both require a labeled set of which items actually count as relevant to a given query.

Mean reciprocal rank scores how early the first relevant result appears, averaged across queries, and is a natural fit when a user typically only needs one correct answer. Normalized discounted cumulative gain generalizes this to graded relevance, where some results are more relevant than others rather than simply relevant or not, discounting a relevant result's contribution the further down the ranking it appears.

Statistical Significance Testing

A metric that improved from one run to the next is not automatically a real improvement; it might be noise from the particular evaluation sample. Significance testing exists to answer a narrower, more useful question: how likely is a gap this large to appear even if the two systems are, in truth, equally good.

Bootstrap resampling estimates a metric's uncertainty directly from the evaluation data itself, by resampling it with replacement many times and recomputing the metric on each resample, rather than assuming the metric follows a particular textbook distribution.

McNemar's test is built specifically for comparing two classifiers on the same test set, looking only at the examples where the two systems disagreed rather than at every example, which is the more statistically appropriate comparison when both systems were scored on identical data.

Common Evaluation Pitfalls

Data leakage happens when information that would not be available at real prediction time slips into training, often through a feature derived from the future or from the label itself. A model with leaked features can score extremely well in evaluation and fail immediately in production, where that information was never actually available.

Test set reuse is a slower version of the same problem: repeatedly tuning a model against the same held out set, even without ever training on it directly, lets a team unconsciously overfit their design choices to that one sample.

Goodhart's law, that a measure which becomes a target stops being a good measure, applies directly to model evaluation. Optimizing hard against one published metric tends to find the cracks in that metric rather than genuine improvement, which is the core argument for publishing several metrics side by side instead of one headline number.

Evaluating Large Language Models

Open-ended text generation breaks the exact-match assumption most classical metrics quietly depend on: two answers can use entirely different words and still be equally correct, so scoring a generated answer usually means scoring something softer than string equality, whether that is embedding similarity, a rubric, or a second model acting as a judge.

Using a large language model to judge another model's output, commonly called LLM-as-judge, is fast and cheap compared to human review, and it inherits real blind spots: a documented tendency toward self-preference bias, where a judge rates its own model family's answers more favorably independent of actual quality, and a documented position bias, where the order two answers are presented in shifts which one a judge prefers.

Faithfulness and hallucination are the two headline concerns for a system that generates answers grounded in retrieved source text. Faithfulness checking is most credible when it comes from a method independent of the model that generated the answer in the first place, since a model asked to grade its own output carries the same blind spots that produced any error to begin with.

Human Evaluation and Annotation

Any evaluation grounded in human labels is only as good as the guidelines those labels were produced under. Inter-annotator agreement, how consistently independent labelers reach the same judgment on the same item, is the practical ceiling on how well any model can be expected to match a single ground truth, and a low agreement score usually points to an ambiguous task definition rather than to careless annotators.

Active learning targets a limited labeling budget at the examples most likely to be informative, typically the ones a current model is least confident about, rather than labeling a uniform random sample. This can meaningfully reduce the number of labels needed to reach a given evaluation precision, at the cost of a labeled set that is no longer a simple random sample of the underlying distribution.

Online Versus Offline Evaluation

An offline metric, computed against a fixed, already collected dataset, is cheap to rerun and easy to reproduce, but it is only ever a proxy for how a system will actually behave in front of real users. A model that wins offline can still lose online, most often because the offline dataset does not reflect the live traffic distribution.

Online evaluation, typically an A/B test against live traffic, measures real behavior directly but costs real user exposure and takes real time to reach statistical significance. A guardrail metric, a secondary number tracked alongside the primary one specifically to catch a regression the primary metric would not notice on its own, such as latency tracked next to answer quality, is standard practice precisely because optimizing one number in isolation tends to find that number's blind spots.

Fairness and Subgroup Evaluation

An aggregate metric can look strong while hiding a badly performing subgroup entirely; subgroup slicing, breaking the same evaluation down by demographic group, input length, or any other relevant split, is the direct way to check for this rather than assume it away.

Demographic parity and equalized odds are two common, and sometimes mutually incompatible, definitions of a fair outcome: the first asks for an equal positive prediction rate across subgroups regardless of the true outcome rate, and the second asks for equal error rates instead. Which definition is appropriate depends on the decision the model is actually informing, not on which one is easier to satisfy.

Evaluation for Agentic and Tool-Using Systems

A system that takes several steps, calling tools and making decisions along the way, cannot be fully judged by its final answer alone. Two runs that reach the same correct answer are not equally good if one of them took a direct path and the other burned through retries, called the wrong tool twice, or recovered from a mistake by luck rather than by a sound decision.

Task success rate, whether the run reached a correct final state at all, remains the single most load-bearing number for an agentic system, but it should be reported alongside step efficiency, how many actions the run took relative to a reasonable minimum, and error recovery rate, how often the system noticed and corrected its own mistake rather than compounding it.

Trajectory level evaluation, scoring the full sequence of steps rather than only the outcome, surfaces failure modes an outcome-only metric hides entirely: a system that reaches the right answer for the wrong reason on nine tasks will look identical to a genuinely reliable one until the tenth task's reasoning happens to matter.

Benchmark Design

A benchmark is only informative for as long as it stays uncontaminated: once a benchmark's questions and answers appear in a model's pretraining data, a high score on it stops distinguishing genuine capability from memorization. This risk has grown as pretraining corpora increasingly scrape the open web, where a popular benchmark is likely to already be discussed, and sometimes solved, somewhere in it.

Benchmark saturation, where every system under comparison scores close to the maximum, is a sign the benchmark has stopped being able to distinguish systems, not a sign every system has gotten equally good. A saturated benchmark needs to be retired or extended with harder cases, not treated as evidence the underlying problem is solved.

A production evaluation set and a public benchmark serve different purposes and should not be treated as substitutes for each other: a benchmark supports comparison against the wider field, while a production evaluation set, built from the actual distribution of real requests, is what predicts how a system will behave for real users.

Multi-Metric Tradeoffs

Two systems rarely agree on every metric at once, and collapsing several metrics into a single weighted score hides the tradeoff a reader most needs to see: which system wins depends entirely on how much each metric is weighted, and that weighting is a judgment call the report is making silently on the reader's behalf.

Reporting a Pareto frontier, the set of configurations where no other configuration is strictly better on every metric at once, lets a reader apply their own priorities instead of inheriting the author's. A configuration that loses on every individual metric compared to some other configuration can be safely dropped from the frontier; everything left on it is a genuine, defensible tradeoff.

A/B Testing Pitfalls

Checking an A/B test's result before it reaches its planned sample size, and stopping as soon as the result looks favorable, inflates the false positive rate far beyond the test's stated significance threshold. This peeking problem is one of the most common ways a team convinces itself of a result that later fails to replicate.

Running many metrics, or many variants, in the same test multiplies the chance that at least one comparison looks significant by pure chance, the multiple comparisons problem. A single headline metric decided in advance, with everything else reported as supporting evidence rather than as an independent test, keeps this risk contained.

A novelty effect, where users respond to a change simply because it is different rather than because it is better, can make a short test look like a clear win that fades once the change stops being new. A sample ratio mismatch, where traffic did not actually split the way the test design assumed, is a data quality check worth running before trusting any result at all.

Synthetic and LLM-Generated Evaluation Data

Using a language model to generate evaluation questions, rather than writing every one by hand, can scale an evaluation set far faster than manual authoring allows. It carries a specific risk: a generator built from the same family of model as the system under test can share its blind spots, producing questions that are easy for exactly the reasons the system under test is already strong.

A human-reviewed subset of any synthetically generated evaluation set is what catches this before it becomes invisible: at minimum, a sample large enough to estimate what fraction of the generated questions are actually well formed, correctly labeled, and free of the generator's own systematic blind spots.

Versioning and Reproducibility

The same metric name can silently mean different computations across two teams, or even across two versions of the same codebase: a recall at five computed against a different candidate pool size, a different tie-breaking rule, or a different definition of what counts as a match will produce a different number for the same underlying system.

Pinning the exact model version, the exact evaluation set version, and the exact evaluation code, ideally down to a specific commit, is what makes a published number reproducible rather than merely stated. A metric that cannot be recomputed by someone else from a documented starting point is closer to a claim than to a measurement.

Random seeds matter more than they are usually given credit for in any evaluation with a stochastic component, from data shuffling to sampling-based generation. Reporting a result across several seeds, rather than the single seed that happened to run first, is a cheap way to know whether a result is stable or was partly a product of that one run.

Cost, Latency, and Efficiency

Quality metrics on their own describe a system that can be built without ever asking what it costs to run. Latency and cost per request are not secondary concerns tacked onto an evaluation; a system too slow or too expensive to run at the traffic it needs to serve is not actually a solution, however strong its offline quality numbers look.

Latency is usually reported as a distribution rather than a single average, most often as p50 and p99: the median request and the request at the ninety-ninth percentile. The two numbers answer different questions, and a system with a fast median but a slow p99 will feel unreliable to a meaningful share of real users even though its average looks fine.

Quality and cost typically trade off against each other along a curve rather than at a single point, and the honest way to report that tradeoff is the whole curve, not the single configuration that happened to win on the headline metric. A reader deciding whether a system fits their own latency or budget constraint needs the curve to make that call themselves.

Reporting Results Honestly

A single point estimate invites more confidence than it deserves. Reporting a confidence interval alongside a metric, even a rough bootstrap based one, tells a reader how much of an observed difference could plausibly be noise from the particular evaluation sample rather than a real gap between systems.

Evaluating on the same split repeatedly while iterating on a system, then reporting only the configuration that scored highest on that split, quietly reproduces the same problem test set contamination causes: the reported number stops describing how the system performs on new data and starts describing how well it was tuned to one particular sample.

A methodology section that states exactly how a number was computed, what the evaluation set contains, and what was excluded or cut for time is what lets another engineer trust a result enough to build on it. A result without a stated methodology is not more impressive for being unqualified; it is simply not verifiable.

Dataset Size and Statistical Reliability

A metric computed against too small an evaluation set carries more uncertainty than its single reported value lets on. Margin of error on a proportion shrinks with the square root of the sample size, not linearly, which is why doubling an evaluation set's size only buys a modest tightening of the result, not a doubling of confidence in it.

A category broken out within a larger evaluation set, such as one failure mode among several being tracked separately, inherits whatever sample size actually lands in that category, not the full set's size. A headline evaluation set that sounds large can still carry a category represented by only a handful of examples, and a pass rate computed from a handful of examples deserves to be reported with that caveat attached.

There is no single correct minimum sample size that applies everywhere; it depends on how large a difference actually needs to be detected and how much uncertainty a decision downstream of the metric can tolerate. What matters in practice is simply stating the category counts a report's numbers are actually built on, so a reader can judge that for themselves rather than assume every category is equally well supported.

Evaluating Systems With No Ground Truth

Some tasks have no single correct answer to score against at all: a creative writing system, an open-ended chat assistant, or a summarizer where several summaries could reasonably be considered equally good. Absolute scoring against a fixed reference breaks down here, since the reference itself is only one of many acceptable outputs.

Pairwise preference judging, asking which of two outputs is better rather than scoring either one in isolation, tends to produce more consistent judgments than absolute scoring for exactly this kind of task, whether the judge is a human rater or a language model standing in for one.

Aggregating many pairwise comparisons into a single ranking, using an Elo-style rating system borrowed from competitive games, turns a large set of head to head preferences into one comparable number per system, and is now common practice for ranking open-ended language model outputs at scale.

Interpreting a Negative Result

A new system that fails to beat its baseline is still a result worth publishing, not a failed experiment to quietly discard. A negative result rules something out, and a field that only ever publishes wins ends up with a literature that systematically overstates how often a given kind of change actually helps.

A result that is not statistically significant is not the same claim as evidence of no difference; it may simply reflect an evaluation set too small to detect the size of difference that is actually present. Reporting the confidence interval, not only whether it happened to cross zero, is what lets a reader tell these two cases apart.

A Pre-Publication Checklist

Before a metric goes into a report, a few questions are worth asking directly rather than assuming the answer. Was the evaluation set ever used to make a design decision during development, which would make it a validation set wearing a test set's label. Is the reported number a point estimate alone, or does it carry some sense of the uncertainty behind it.

Was the result checked at the subgroup level, not only in aggregate. Is the exact model version, evaluation set version, and evaluation code pinned well enough that someone else could reproduce the number from what was published. Does the report name what was cut or left unmeasured, rather than staying silent about it.

None of these questions has a single correct answer that applies to every project; a one day build and a year long research effort can reasonably draw the line in different places. What matters is that the line was drawn on purpose and stated plainly, rather than left for a reader to assume the most generous interpretation of.

Choosing a Significance Test

The right significance test depends on the specific question being asked, not on habit. The table below is a starting point for the most common comparisons that come up when evaluating machine learning systems, not an exhaustive statistics reference.

Question being asked	A reasonable test to reach for
Are two classifiers different on the same test set?	McNemar's test
Is a metric's mean different from a fixed target value?	One-sample t-test
Are two independent systems' mean metrics different?	Two-sample t-test
What is the uncertainty on a single reported metric?	Bootstrap confidence interval
Are two categorical distributions different?	Chi-square test
Do rankings from two judges agree?	Spearman or Kendall rank correlation

Metric Reference Table

The table below collects every metric named in this guide, plus several common ones from adjacent tasks, in one place for quick lookup.

Metric	Category	Range	Better	Typical use
Accuracy	Classification	[0, 1]	Higher	Overall correctness on balanced classes
Precision	Classification	[0, 1]	Higher	Cost of a false positive is high
Recall	Classification	[0, 1]	Higher	Cost of a false negative is high
F1 Score	Classification	[0, 1]	Higher	Balance precision and recall
F-beta Score	Classification	[0, 1]	Higher	Weight recall over precision, or the reverse
Specificity	Classification	[0, 1]	Higher	True negative rate, screening tests
ROC-AUC	Classification	[0, 1]	Higher	Threshold independent ranking quality
PR-AUC	Classification	[0, 1]	Higher	Imbalanced positive class
Log Loss	Classification	[0, inf)	Lower	Penalizes confident wrong predictions
Brier Score	Classification	[0, 1]	Lower	Calibration of predicted probabilities
Matthews Correlation Coefficient	Classification	[-1, 1]	Higher	Balanced measure on imbalanced classes
Cohen's Kappa	Classification	[-1, 1]	Higher	Agreement beyond chance
Balanced Accuracy	Classification	[0, 1]	Higher	Average recall across imbalanced classes
Mean Absolute Error	Regression	[0, inf)	Lower	Interpretable average error magnitude
Mean Squared Error	Regression	[0, inf)	Lower	Penalize large errors more
Root Mean Squared Error	Regression	[0, inf)	Lower	Same units as the target
Mean Absolute Percentage Error	Regression	[0, inf)	Lower	Relative error across scales

R-squared	Regression	$(-\infty, 1]$	Higher	Variance explained versus a mean baseline
Adjusted R-squared	Regression	$(-\infty, 1]$	Higher	R-squared penalized for extra features
Huber Loss	Regression	$[0, \infty)$	Lower	Robust to outliers
Precision at k	Ranking	$[0, 1]$	Higher	Fraction of the top k results that are relevant
Recall at k	Ranking	$[0, 1]$	Higher	Fraction of relevant results found in the top k
Mean Reciprocal Rank	Ranking	$[0, 1]$	Higher	How early the first relevant result appears
Mean Average Precision	Ranking	$[0, 1]$	Higher	Precision averaged across relevant ranks
Normalized Discounted Cumulative Gain	Ranking	$[0, 1]$	Higher	Graded relevance with a position discount
Hit Rate at k	Ranking	$[0, 1]$	Higher	Whether any relevant result appears in the top k
Expected Calibration Error	Calibration	$[0, 1]$	Lower	Gap between confidence and accuracy
Maximum Calibration Error	Calibration	$[0, 1]$	Lower	Worst case calibration gap across bins
Perplexity	Language Modeling	$[1, \infty)$	Lower	How well a model predicts held out text
BLEU Score	Language Generation	$[0, 1]$	Higher	N-gram overlap with reference text
ROUGE-L	Language Generation	$[0, 1]$	Higher	Longest common subsequence overlap
Word Error Rate	Speech Recognition	$[0, \infty)$	Lower	Edit distance over reference words
Intersection over Union	Computer Vision	$[0, 1]$	Higher	Overlap between predicted and true regions

Mean Average Precision at IoU	Computer Vision	[0, 1]	Higher	Detection quality across IoU thresholds
Silhouette Score	Clustering	[-1, 1]	Higher	Separation between clusters
Davies-Bouldin Index	Clustering	[0, inf)	Lower	Average similarity to the closest other cluster
Adjusted Rand Index	Clustering	[-1, 1]	Higher	Agreement with a ground truth partition
Coverage at k	Recommenders	[0, 1]	Higher	Fraction of the catalog ever recommended
Novelty	Recommenders	[0, inf)	Higher	How unexpected recommended items are
Diversity	Recommenders	[0, 1]	Higher	Dissimilarity among items in one recommendation list
BERTScore	Language Generation	[0, 1]	Higher	Embedding similarity to a reference, not exact words
Faithfulness Rate	RAG	[0, 1]	Higher	Share of claims entailed by their cited source
Hallucination Rate	RAG	[0, 1]	Lower	Share of claims contradicted by or absent from the source
Groundedness	RAG	[0, 1]	Higher	Whether an answer is traceable to retrieved evidence at all
Win Rate	Preference Evaluation	[0, 1]	Higher	Share of paired comparisons a system's output is preferred in
Toxicity Rate	Safety	[0, 1]	Lower	Share of outputs flagged as toxic or harmful by a classifier
Refusal Rate	Safety	[0, 1]	Varies	Share of prompts a system declines to answer

Glossary

Baseline. A simple reference model or rule used to judge whether a more complex model's performance is actually worth the added complexity.

Confusion Matrix. A table cross tabulating predicted classes against actual classes, the source most classification metrics are computed from.

Cross-Validation. Splitting data into multiple folds and rotating which fold is held out for evaluation, so a reported metric does not depend on one lucky split.

Data Leakage. Information from outside the training set, often from the future or from the label itself, that lets a model perform well during evaluation without truly generalizing.

Ground Truth. The accepted correct label or answer a prediction is evaluated against.

Held-Out Set. Data never used during training or model selection, reserved specifically to estimate real world performance.

Hyperparameter. A configuration value set before training rather than learned from data, such as a learning rate or the number of trees in a forest.

Inter-Annotator Agreement. How consistently multiple human labelers agree with each other, which puts an upper bound on how well any model can be expected to match a single ground truth.

Overfitting. A model that has learned patterns specific to its training data rather than patterns that generalize, visible as strong training performance and weak held out performance.

Stratified Sampling. Splitting data so that each split preserves the same class proportions as the full dataset.

Test Set. Data held out for a single, final evaluation, not used to tune the model at all.

Threshold. The cutoff probability above which a model's output is treated as a positive prediction.

Training Set. The data a model directly learns its parameters from.

Underfitting. A model too simple to capture the real pattern in the data, visible as weak performance on both training and held out data.

Validation Set. Data used to tune hyperparameters and select among models, kept separate from the final test set.

Weak Label. A label produced by an imperfect, automated, or indirect process rather than careful human review.

Bootstrap Resampling. Repeatedly resampling a dataset with replacement to estimate how much a metric would vary across different samples of the same population.

Calibration. How closely a model's predicted probabilities match the true frequency of the outcome they describe.

Class Imbalance. A dataset where one label is far more common than another, which can make accuracy a misleading metric.

Confidence Interval. A range expected to contain the true value of a metric with some stated probability, given the sampling variation in the evaluation data.

Distribution Shift. A change between the data a model was evaluated on and the data it later encounters in production.

Effect Size. How large a difference actually is, independent of whether it is statistically significant.

Gold Standard. The highest quality available reference labels, typically produced through careful, well specified human review.

Null Hypothesis. The default assumption in a significance test, usually that two systems perform the same, which the test looks for evidence against.

P-value. The probability of seeing a difference at least this large if the null hypothesis were actually true.

Statistical Power. The probability that a significance test correctly detects a real difference when one actually exists.

Active Learning. Choosing which unlabeled examples to label next, usually the ones a model is least confident about, to get the most value from a limited annotation budget.

Annotation Guidelines. The written instructions that define how a labeling task should be judged, which a disagreement between two annotators can usually be traced back to a gap in.

Demographic Parity. A fairness criterion requiring a model's positive prediction rate to be equal across subgroups, regardless of whether the true outcome rate actually differs between them.

Equalized Odds. A fairness criterion requiring a model's true positive and false positive rates to be equal across subgroups, a different and sometimes incompatible standard from demographic parity.

Guardrail Metric. A secondary metric tracked alongside the primary one specifically to catch a regression the primary metric would not notice, such as latency next to accuracy.

LLM-as-Judge. Using a large language model to score another system's output, a fast and cheap approach that inherits that judge model's own blind spots, including a documented tendency to prefer its own family's outputs.

Offline Evaluation. Evaluation run against a fixed, pre-collected dataset rather than against live traffic, cheap to repeat but only ever a proxy for how a system behaves with real users.

Online Evaluation. Evaluation run against live traffic, typically through an A/B test, which measures real user behavior but costs real exposure and takes real time to reach significance.

Self-Preference Bias. A judge model's tendency to rate outputs from its own model family more favorably, independent of actual quality, which is why an independent judge is preferred for a high stakes comparison.

Subgroup Slicing. Breaking an aggregate evaluation metric down by subgroup, such as by demographic group or by input length, since a strong aggregate number can hide a badly performing slice.

Closing Note

None of the metrics or tests collected here substitute for the harder work of deciding what actually matters for a given system before measuring it. A well chosen metric reported honestly, with its uncertainty and its blind spots stated plainly, is worth more than a longer table of numbers nobody stopped to interpret.

This guide was written as original reference material for a document-grounded retrieval project, specifically to give that project's retrieval and chunking comparison something structurally different from its own fictional demo business to run against: a different domain, a real multi-page table, and a real two column section, none of which its other demo documents have.